Module 05 — LLM Serving & Inference Engines

> **Goal:** Choose, configure, benchmark and operate a production inference stack. Understand PagedAttention, continuous batching, chunked prefill, speculative decoding and multi-LoRA serving well enough to derive their effects from first principles.

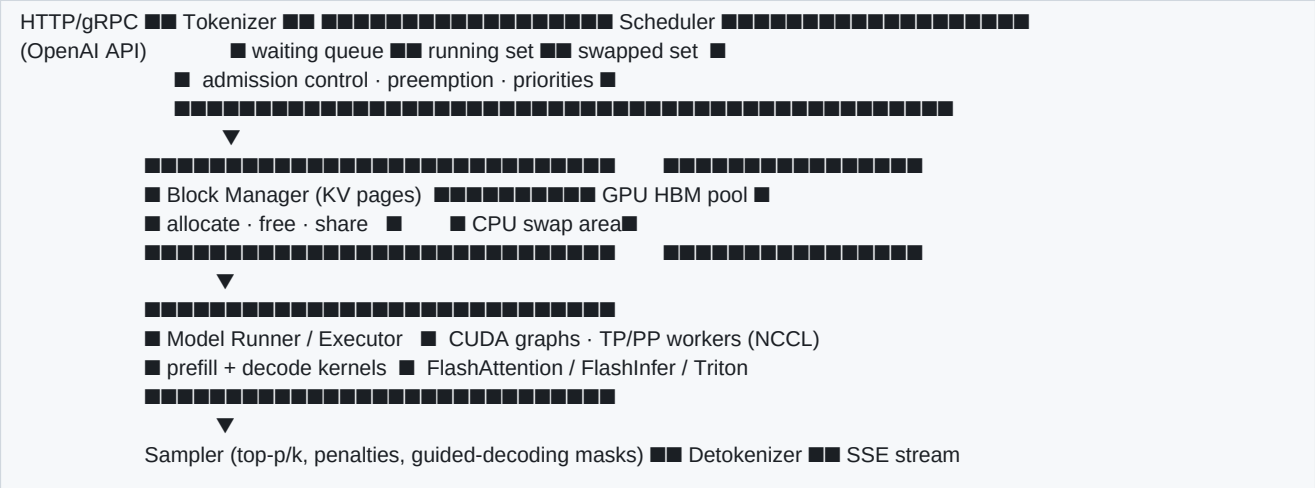
Table of Contents

1. Module Overview & First-Principles Architecture
2. Mathematical Foundations
3. Production-Ready Code
4. Hard-Learned Production Failures
5. Interview Questions

---

1. Module Overview & First-Principles Architecture

1.1 Anatomy of an inference server



1.2 Engine comparison

Engine	Strengths	Weaknesses	Best fit
vLLM	PagedAttention, continuous batching, prefix caching, chunked prefill, multi-LoRA, guided decoding, speculative decoding, broad model/quant support, OpenAI-compatible, fast-moving community	Python scheduler overhead at extreme QPS; version churn	Default for general production serving

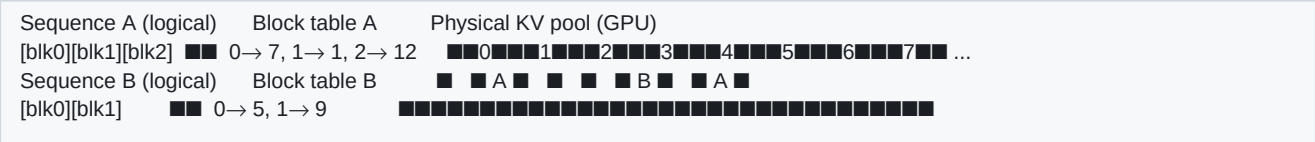
Engine	Strengths	Weaknesses	Best fit
TensorRT-LLM (+ Triton / NVIDIA Dynamo)	Highest peak throughput/lowest latency on NVIDIA via compiled engines, in-flight batching, FP8/FP4 on Hopper/Blackwell, paged KV, speculative decoding	Engine build step per GPU arch/shape; complex ops; NVIDIA-only	Large fixed deployments where 20–40 % extra efficiency pays for engineering
HF TGI	Mature, Rust router, good HF-hub integration, flash/paged attention, tensor parallel, quantization, Prometheus metrics	Licence/maintenance mode changes; fewer cutting-edge features than vLLM	Teams on HF Inference Endpoints; simple deployments
Ollama / llama.cpp	Trivial setup, CPU/Metal/CUDA, GGUF, great for dev & edge	Low concurrency; limited batching; not for multi-tenant high QPS	Local dev, laptops, edge
SGLang	RadixAttention (prefix tree cache), fast structured decoding, strong for agentic multi-call workloads	Smaller ecosystem	Prefix-heavy, structured-output workloads

Decision shortcut: **dev** → **Ollama; default prod** → **vLLM; max efficiency at scale on NVIDIA** → **TensorRT-LLM; heavy prefix sharing + structured** → **consider SGLang.** Always benchmark with *your* traffic.

1.3 PagedAttention — virtual memory for KV cache

Problem (pre-vLLM). Frameworks reserved a contiguous KV buffer of `max_seq_len` per request. Because output length is unknown, that wasted 60–80 % of KV memory via *internal fragmentation* (reserved but unused) and *external fragmentation* (gaps between buffers).

Idea. Split KV cache into fixed-size **blocks** (e.g., 16 tokens). Each sequence has a **block table** mapping logical block index → physical block. Blocks are allocated on demand as tokens are generated.



Benefits: 1. **Near-zero fragmentation** (waste ≤ one partially-filled block per sequence) → 2–4× more concurrent sequences. 2. **Copy-on-write sharing:** parallel sampling / beam search / shared prompts point to the same physical blocks; only diverging blocks are copied. 3. **Prefix caching:** blocks are hashed by `(parent hash, token ids)`; identical prefixes across requests reuse the physical blocks (system prompts, few-shot, documents). 4. **Preemption** by swapping blocks to CPU or recomputing.

The attention kernel is modified to gather K/V through the block table (non-contiguous reads), costing a few % kernel efficiency but enabling much larger batches.

1.4 Continuous (iteration-level) batching

Static batching waits for a full batch and runs until the *longest* sequence finishes → short requests are held hostage and the GPU idles on padding.

Continuous batching (Orca): scheduling is per **iteration**. After each decode step finished sequences leave and waiting ones join (their prefill is merged in the next step). GPU utilisation and throughput rise 5–20× vs static batching; tail latency falls because short requests exit early.

```
step:  1  2  3  4  5  6  7  8
slot0: A  A  A  A  E  E  E  E    (A done at 4, E joins at 5)
slot1: B  B  C  C  C  C  F  F    (B done at 2, C joins at 3, ...)
slot2: D  D  D  D  D  D  D  G
```

1.5 Chunked prefill

A 32k-token prefill monopolises compute for ~0.5–2 s, stalling all running decodes (ITL spikes → "stutter" for every user) — **head-of-line blocking**. Chunked prefill splits the prompt into chunks (e.g., 2048 tokens) and *piggybacks decode tokens* in the same batch (Sarathi-Serve "stall-free batching"). Effects: smoother ITL, bounded activation memory, slightly higher TTFT for the long prompt itself. Tune `max_num_batched_tokens` (chunk budget): larger → better throughput/TTFT for prefills, worse ITL; smaller → better ITL.

1.6 Speculative decoding

Decode is memory-bound, so verifying `k` tokens in one forward pass costs about as much as generating one. A cheap **draft** proposes `k` tokens; the **target** verifies them in parallel; accepted prefix is kept; the first rejected token is resampled from a corrected distribution. The output distribution is *identical* to the target's (lossless).

Variants: separate draft model (e.g., 1B for 70B), **n-gram / prompt-lookup** (no model; great for summarisation/RAG/code editing where output copies input), **EAGLE/EAGLE-3 and Medusa heads** (small trained heads on target hidden states), **MTP** (multi-token prediction heads built into the model, e.g., DeepSeek).

Speculation shines at **low batch / latency-sensitive** workloads; at high batch the GPU is already compute-saturated and the extra verification FLOPs reduce throughput.

1.7 Multi-LoRA serving

One base model resident in memory + many small adapters. For each request in a batch, the LoRA delta is applied with segmented/batched GEMM kernels (SGMV/BGMV from Punica / S-LoRA). Memory: base 16 GB + N × (~100–300 MB). Adapter hot-loading from CPU/disk on demand with an LRU. This yields per-tenant customisation at near-single-model cost.

1.8 Parallelism choices

Strategy	Splits	Communication	Use when
Tensor parallel (TP)	each layer's matrices across GPUs	all-reduce *every layer* (2/layer) → needs NVLink	model doesn't fit on 1 GPU; latency matters
Pipeline parallel (PP)	layers across GPUs/nodes	point-to-point activations between stages	cross-node over slower links; throughput focus; bubbles at small batch
Data parallel / replicas	whole model copies	none	scale throughput; the simplest
Expert parallel (EP)	MoE experts across GPUs	all-to-all	MoE models (Mixtral, DeepSeek-V3, Qwen-MoE)
Disaggregated prefill/decode	phases on different pools	KV transfer (NIXL/NVLink/RDMA)	strict TTFT *and* ITL SLOs at scale

1.9 Disaggregated serving

Prefill (compute-bound) and decode (memory-bound) interfere. Splitting them lets each pool be sized and tuned independently (prefill: high TP, large batch tokens; decode: many replicas, high batch). Cost: KV-cache transfer over fast interconnect; complexity. Frameworks: vLLM disaggregated prefill, NVIDIA Dynamo, llm-d, Mooncake.

2. Mathematical Foundations & Formulas

2.1 KV block geometry

```
bytes_per_block = block_size × 2 × L × n_kv × d_h × b_kv / TP
num_gpu_blocks  = floor( (GPU_mem × util – weights – peak_activation – overhead) / bytes_per_block )
max_tokens_cached = num_gpu_blocks × block_size
```

Llama-3.1-8B BF16, block 16: `bytes_per_block = 16 × 131072 = 2 MiB`. With 50 GiB pool → 25,600 blocks = 409,600 tokens.

2.2 Fragmentation comparison

Contiguous reservation: waste ≈ `(S_max – S_actual) / S_max` per sequence — with `S_max = 4096`, mean `S_actual = 600` → 85 % waste. Paged: waste ≤ `block_size – 1` tokens/sequence → `≤ 15/600 = 2.5 %`.

Concurrency gain ≈ `(1 – 0.025) / (1 – 0.85) ≈ 6.5×` for that workload (best case).

2.3 Throughput vs. batch size (roofline)

Decode step time (weights W bytes, KV bytes per sequence K_i, bandwidth BW, compute P):

```
t_step(b) ≈ max( (W + Σ_i K_i) / BW ,  (2·N·b) / P )
tokens/s(b) = b / t_step(b)
```

Knee batch `b* ≈ W·P / (2·N·BW) = (bytes/param)·P / (2·BW)`. For BF16 on H100: `2 × 989e12 / (2 × 3.35e12) ≈ 295`. Below `b*`, throughput scales ~linearly with `b` while per-user ITL stays ~flat — the "free lunch" of batching. KV reads make `t_step` grow with `b` and context length, so the real knee is lower.

2.4 Little's Law for capacity

```
L = λ · W      (avg concurrent requests = arrival rate × avg time in system)
```

If RPS = 20 and mean E2E = 6 s → 120 concurrent sequences. Divide by per-GPU max concurrent sequences (Module 01 §3.2) to get GPU count; add headroom for p99 and failover.

2.5 Speculative decoding speedup

Per-token acceptance probability α (assumed i.i.d.), draft length γ, cost ratio c = `t_draft/t_target`:

```
E[accepted tokens per verification] = (1 – α^{γ+1}) / (1 – α)
Speedup S = (1 – α^{γ+1}) / ((1 – α)(γ·c + 1))
```

Example: α=0.8, γ=4, c=0.05 → numerator `(1–0.328)/0.2 = 3.36`; denominator `0.2+1 = 1.2` → **S ≈ 2.8×**. With α=0.5, γ=4, c=0.1: `(1–0.031)/0.5=1.94`, `/1.4` = **1.38×**. Poor acceptance wastes verification compute.

2.6 Chunked prefill scheduling

Per-iteration token budget `T_b = max_num_batched_tokens`. Decodes consume 1 token each; the remainder `T_b – n_decode` goes to prefill chunks. Iteration time `t_iter ≈ (T_b × 2N)/P_eff` (compute-bound) → **ITL lower bound ≈ t_iter**. To meet ITL SLO of 40 ms on 8B/H100: `T_b ≤ 0.04 × 0.5 × 989e12 / (2 × 8e9) ≈ 1,236 tokens`... in practice kernels at this size aren't perfectly efficient; benchmark between 512–4096.

2.7 TTFT under queueing (M/M/1 approximation)

```
W_q = ρ / (μ(1 − ρ))          ρ = λ/μ
```

At ρ=0.9 the mean queueing delay is 9× the service time; at ρ=0.95, 19×. This is why p95 TTFT collapses before the GPU "looks" saturated; target ρ ≤ 0.7 for latency-sensitive tiers and use admission control/priority classes.

2.8 TP communication cost

All-reduce volume per layer ≈ `2 × (TP−1)/TP × b_tokens × d × bytes`; two per layer. On NVLink (900 GB/s on H100) trivial; over PCIe Gen4 (~25 GB/s effective) it dominates decode: `L=80 layers × 2 × (b × 8192 × 2 B)` per step. For b=64: `80×2×1 MiB ≈ 168 MB` → 6.7 ms at PCIe vs 0.2 ms at NVLink. **Rule: TP across PCIe-only GPUs is usually a mistake; use PP or fewer GPUs with quantization.**

2.9 Latency decomposition

```
E2E = T_net + T_queue + T_tokenize + T_prefill(S_in, chunks) + (S_out − 1)·ITL + T_detok
Goodput = (# requests with TTFT ≤ SLO_ttft ∧ ITL_p95 ≤ SLO_itl) / time
```

---

3. Production-Ready Executable Code

3.1 vLLM Single-Node Production Launch (annotated)

```
#!/usr/bin/env bash
# scripts/serve_vllm.sh — Llama-3.1-8B-Instruct, 1×H100, multi-LoRA, prefix cache, chunked prefill.
set -euo pipefail

export VLLM_USE_V1=1                    # V1 engine: lower CPU overhead, async scheduling
export HF_HOME=/models/hf
export VLLM_ALLOW_RUNTIME_LORA_UPDATING=True

exec vllm serve meta-llama/Llama-3.1-8B-Instruct \
  --served-model-name base \
  --host 0.0.0.0 --port 8000 \
  --dtype bfloat16 \
  --max-model-len 16384 \
  --gpu-memory-utilization 0.90 \
  --max-num-seqs 128 \
  --max-num-batched-tokens 4096 \
  --enable-chunked-prefill \
  --enable-prefix-caching \
  --block-size 16 \
  --kv-cache-dtype fp8 \
  --enable-lora --max-loras 4 --max-lora-rank 64 --max-cpu-loras 32 \
  --lora-modules support=/adapters/support-v15 sql=/adapters/sql-v3 \
  --guided-decoding-backend xgrammar \
  --enable-auto-tool-choice --tool-call-parser llama3_json \
  --api-key "${VLLM_API_KEY:?set VLLM_API_KEY}" \
  --otlp-traces-endpoint http://otel-collector:4317 \
  --collect-detailed-traces all
```

Flag cheat-sheet:

Flag	Effect	Tuning heuristic
`--gpu-memory-utilization`	fraction of HBM vLLM may claim (weights + KV + activations)	0.88–0.92; lower if other processes share the GPU

Flag	Effect	Tuning heuristic
--max-num-seqs	max concurrent sequences per step	≤ KV capacity / avg ctx; 64–256
--max-num-batched-tokens	token budget per iteration (chunk size)	2048–8192; lower → smoother ITL
--max-model-len	max prompt+output	smaller values free KV for batch; set to p99.9 of real needs
--enable-prefix-caching	reuse KV of shared prefixes	on for RAG/chat with static system prompts
--kv-cache-dtype fp8	halve KV memory	validate long-context quality
--swap-space	CPU GiB for swapped sequences	4–16; or rely on recompute
--tensor-parallel-size	TP degree	smallest that fits + headroom; NVLink only
--speculative-config	speculative decoding	low-batch latency tiers

3.2 Tensor-Parallel 70B on 4×H100 (FP8) with Speculative Decoding

```
vllm serve meta-llama/Llama-3.3-70B-Instruct \
  --quantization fp8 --kv-cache-dtype fp8 \
  --tensor-parallel-size 4 \
  --max-model-len 32768 --max-num-seqs 64 --max-num-batched-tokens 8192 \
  --gpu-memory-utilization 0.92 \
  --enable-prefix-caching --enable-chunked-prefill \
  --speculative-config '{"method":"ngram","num_speculative_tokens":4,"
prompt_lookup_max":4}'
# Model-based draft alternative:
#   --speculative-config '{"model":"meta-llama/Llama-3.2-1B-Instruct","num_specu
lative_tokens":4}'
```

3.3 Docker Compose (GPU) for vLLM + Gateway

```
# docker-compose.serving.yml
services:
  vllm:
    image: vllm/vllm-openai:v0.10.1
    runtime: nvidia
    ipc: host                    # needed for NCCL/shared memory
    environment:
      - HUGGING_FACE_HUB_TOKEN=${HF_TOKEN}
      - VLLM_API_KEY=${VLLM_API_KEY}
    command: >
      --model meta-llama/Llama-3.1-8B-Instruct
      --served-model-name base
      --max-model-len 16384 --gpu-memory-utilization 0.90
      --enable-prefix-caching --enable-chunked-prefill
      --max-num-seqs 128 --max-num-batched-tokens 4096
    volumes: ["hf-cache:/root/.cache/huggingface"]
    ports: ["8000:8000"]
    deploy:
      resources:
        reservations:
          devices: [{ driver: nvidia, count: 1, capabilities: [gpu] }]
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 15s
      timeout: 5s
      retries: 20
      start_period: 180s
volumes:
  hf-cache:
```

3.4 Kubernetes Deployment, Service, HPA (KEDA) for vLLM

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: vllm-llama31-8b, labels: { app: vllm } }
spec:
  replicas: 2
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0, maxSurge: 1 } }
  selector: { matchLabels: { app: vllm } }
  template:
    metadata:
      labels: { app: vllm }
      annotations: { prometheus.io/scrape: "true", prometheus.io/port: "8000", promet
heus.io/path: /metrics }
    spec:
      terminationGracePeriodSeconds: 120      # drain in-flight streams
      nodeSelector: { nvidia.com/gpu.product: NVIDIA-H100-80GB-HBM3 }
      tolerations: [{ key: nvidia.com/gpu, operator: Exists, effect: NoSchedule }]
      containers:
        - name: vllm
          image: vllm/vllm-openai:v0.10.1
          args:
            - --model=/models/llama31-8b
            - --served-model-name=base
            - --max-model-len=16384
            - --gpu-memory-utilization=0.90
            - --enable-prefix-caching
            - --enable-chunked-prefill
            - --max-num-seqs=128
          ports: [{ containerPort: 8000, name: http }]
          env:
            - { name: VLLM_API_KEY, valueFrom: { secretKeyRef: { name: vllm-secret, key: api-key } } }
          resources:
            limits: { nvidia.com/gpu: 1, memory: 64Gi, cpu: "8" }
            requests: { nvidia.com/gpu: 1, memory: 48Gi, cpu: "4" }
          startupProbe:  { httpGet: { path: /health, port: http }, failureThreshold: 90, periodSeconds: 1
0 }
          readinessProbe: { httpGet: { path: /health, port: http }, periodSeconds: 5 }
          livenessProbe:  { httpGet: { path: /health, port: http }, periodSeconds: 20, failureThreshold:
6 }
          lifecycle:
            preStop: { exec: { command: ["sleep", "20"] } }   # let LB deregister bef
ore SIGTERM
          volumeMounts:
            - { name: models, mountPath: /models, readOnly: true }
            - { name: shm, mountPath: /dev/shm }
      volumes:
        - name: models
          persistentVolumeClaim: { claimName: model-store-ro }
        - name: shm
          emptyDir: { medium: Memory, sizeLimit: 16Gi }
---
apiVersion: v1
kind: Service
metadata: { name: vllm }
spec:
  selector: { app: vllm }
  ports: [{ port: 80, targetPort: http }]
---
# Autoscale on QUEUE DEPTH / KV usage, not CPU or GPU utilisation.
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: vllm-scaler }
spec:
  scaleTargetRef: { name: vllm-llama31-8b }
```

```
  minReplicaCount: 2
  maxReplicaCount: 12
  cooldownPeriod: 300
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus.monitoring:9090
        query: sum(vllm:num_requests_waiting{app="vllm"}) / count(vllm:num_requests_running{app
="vllm"})
        threshold: "2"
    - type: prometheus
      metadata:
        serverAddress: http://prometheus.monitoring:9090
        query: avg(vllm:gpu_cache_usage_perc{app="vllm"})
        threshold: "0.8"
```

Why not CPU/GPU-util HPA? GPU "utilisation" shows 100 % whenever any kernel is resident, even if mostly idle on bandwidth. Queue depth and KV usage directly measure saturation.

3.5 Helm `values.yaml` (generic chart pattern)

```
replicaCount: 2
image: { repository: vllm/vllm-openai, tag: v0.10.1, pullPolicy: IfNotPresent }
model:
  name: meta-llama/Llama-3.1-8B-Instruct
  servedName: base
  maxModelLen: 16384
engine:
  gpuMemoryUtilization: 0.90
  maxNumSeqs: 128
  maxNumBatchedTokens: 4096
  enablePrefixCaching: true
  enableChunkedPrefill: true
  kvCacheDtype: fp8
  tensorParallelSize: 1
lora:
  enabled: true
  maxLoras: 4
  modules:
    support: /adapters/support-v15
    sql: /adapters/sql-v3
resources:
  limits: { nvidia.com/gpu: 1, memory: 64Gi }
autoscaling:
  enabled: true
  keda:
    minReplicas: 2
    maxReplicas: 12
    queueThreshold: 2
    kvCacheThreshold: 0.8
podDisruptionBudget: { minAvailable: 1 }
serviceMonitor: { enabled: true, interval: 15s }
```

3.6 TensorRT-LLM: Build and Serve (FP8, Llama-3.1-8B)

```
#!/usr/bin/env bash
set -euo pipefail
HF=/models/llama31-8b
CKPT=/engines/ckpt_fp8
ENG=/engines/llama31-8b-fp8-tp1

# 1. Quantize to FP8 with calibration (ModelOpt)
python examples/quantization/quantize.py --model_dir $HF --dtype float16 \
```

```
  --qformat fp8 --kv_cache_dtype fp8 --calib_size 512 --output_dir $CKPT

# 2. Build the engine (shapes are baked in; choose them from real traffic percentiles)
trtllm-build --checkpoint_dir $CKPT --output_dir $ENG \
  --gemm_plugin auto --gpt_attention_plugin auto \
  --max_batch_size 128 --max_input_len 8192 --max_seq_len 12288 --max_num_tokens 8192 \
  --use_paged_context_fmha enable --kv_cache_type paged --workers 1

# 3. Serve via OpenAI-compatible server
trtllm-serve $ENG --tokenizer $HF --host 0.0.0.0 --port 8000 \
  --max_batch_size 128 --max_num_tokens 8192 --kv_cache_free_gpu_memory_fraction 0.9
```

Modern TRT-LLM also offers a PyTorch backend (trtllm-serve <hf_model> --backend pytorch) skipping the explicit build.

3.7 HuggingFace TGI

```
docker run --gpus all --shm-size 1g -p 8080:80 -v $PWD/data:/data \
  -e HF_TOKEN=$HF_TOKEN \
  ghcr.io/huggingface/text-generation-inference:3.3 \
  --model-id meta-llama/Llama-3.1-8B-Instruct \
  --max-input-tokens 8192 --max-total-tokens 12288 \
  --max-batch-prefill-tokens 8192 --max-concurrent-requests 256 \
  --quantize awq   # when serving an AWQ checkpoint
```

3.8 Ollama (dev) with Concurrency Settings

```
OLLAMA_NUM_PARALLEL=4 OLLAMA_MAX_LOADED_MODELS=2 OLLAMA_KEEP_ALIVE=30m O
LLAMA_FLASH_ATTENTION=1 \
OLLAMA_KV_CACHE_TYPE=q8_0 ollama serve
ollama pull llama3.1:8b
curl http://localhost:11434/v1/chat/completions -d '{"model":"llama3.1:8b","mess
ages":[{"role":"user","content":"hi"}]}'
```

3.9 Async Load Generator Measuring TTFT / ITL / TPS / Goodput

```
#!/usr/bin/env python3
"""Open-loop load test against an OpenAI-compatible streaming endpoint.

python bench.py --url http://localhost:8000/v1 --model base --rps 8 --duration 120 \
    --prompt-lens 512,2048,6000 --out-len 256 --slo-ttft 0.7 --slo-itl 0.04
"""
from __future__ import annotations

import argparse
import asyncio
import json
import random
import statistics
import time
from dataclasses import dataclass, field

import httpx


@dataclass
class Result:
    ok: bool
    ttft: float = 0.0
    itls: list[float] = field(default_factory=list)
    tokens: int = 0
    e2e: float = 0.0
```

```
def make_prompt(n_tokens: int) -> str:
    words = ["alpha", "bravo", "charlie", "delta", "echo&quo
t;, "foxtrot", "golf", "hotel"]
    return " ".join(random.choice(words) for _ in range(int(n_tokens * 0.75))) + "\nSummar
ise the above."


async def one(client: httpx.AsyncClient, url: str, model: str, prompt: str, out_len: int) -> Result:
    body = {"model": model, "messages": [{"role": "user", "c
ontent": prompt}], "max_tokens": out_len,
            "stream": True, "temperature": 0.0, "ignore_eos": True}
    t0 = time.perf_counter(); last = t0; first = True; r = Result(ok=False)
    try:
        async with client.stream("POST", f"{url}/chat/completions", json=body, timeou
t=300) as resp:
            if resp.status_code != 200:
                return r
            async for line in resp.aiter_lines():
                if not line.startswith("data: ") or line.endswith("[DONE]"):
                    continue
                delta = json.loads(line[6:])["choices"][0]["delta"].get("content
")
                if not delta:
                    continue
                now = time.perf_counter()
                if first:
                    r.ttft = now - t0; first = False
                else:
                    r.itls.append(now - last)
                last = now; r.tokens += 1
        r.e2e = time.perf_counter() - t0; r.ok = True
    except Exception:
        pass
    return r


def pct(xs: list[float], p: float) -> float:
    if not xs:
        return float("nan")
    xs = sorted(xs); return xs[min(len(xs) - 1, int(p / 100 * len(xs)))]


async def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", required=True); ap.add_argument("--model", required=True
)
    ap.add_argument("--rps", type=float, default=4); ap.add_argument("--duration", ty
pe=int, default=60)
    ap.add_argument("--prompt-lens", default="1024"); ap.add_argument("--out-len
", type=int, default=256)
    ap.add_argument("--slo-ttft", type=float, default=1.0); ap.add_argument("--slo-itl&quo
t;, type=float, default=0.05)
    a = ap.parse_args()
    lens = [int(x) for x in a.prompt_lens.split(",")]
    limits = httpx.Limits(max_connections=2000)
    async with httpx.AsyncClient(limits=limits, headers={"Authorization": "Bearer EMPTY&qu
ot;}) as c:
        tasks = []; t_start = time.perf_counter()
        while time.perf_counter() - t_start < a.duration:      # Poisson arrivals (open loop)
            tasks.append(asyncio.create_task(one(c, a.url, a.model, make_prompt(random.choice(lens)), a.o
ut_len)))
            await asyncio.sleep(random.expovariate(a.rps))
        results = await asyncio.gather(*tasks)
    wall = time.perf_counter() - t_start
    ok = [r for r in results if r.ok]
```

```
    itls = [x for r in ok for x in r.itls]
    good = [r for r in ok if r.ttft <= a.slo_ttft and pct(r.itls, 95) <= a.slo_itl]
    print(json.dumps({
        "requests": len(results), "success": len(ok),
        "ttft_p50": pct([r.ttft for r in ok], 50), "ttft_p95": pct([r.ttft for r in o
k], 95),
        "itl_p50": pct(itls, 50), "itl_p95": pct(itls, 95), "itl_p99": pct(
itls, 99),
        "output_tok_per_s": sum(r.tokens for r in ok) / wall,
        "per_user_tok_per_s_p50": statistics.median([1 / statistics.mean(r.itls) for r in ok if
 r.itls]),
        "goodput_rps": len(good) / wall,
    }, indent=2))


if __name__ == "__main__":
    asyncio.run(main())
```

3.10 Per-request Multi-LoRA and Guided Decoding Client

```
from __future__ import annotations

from openai import OpenAI

client = OpenAI(base_url="http://localhost:8000/v1", api_key="EMPTY")


def ask(adapter: str, question: str) -> str:
    # `model` selects the adapter registered via --lora-modules; "base" = no adapter.
    r = client.chat.completions.create(model=adapter, messages=[{"role": "user", &quo
t;content": question}], temperature=0.2)
    return r.choices[0].message.content or ""


print(ask("support", "How do I rotate my API key?"))
print(ask("sql", "Total revenue per region in 2024 from table sales(region, amount, ts).&q
uot;))
```

Dynamic adapter load at runtime (requires VLLM_ALLOW_RUNTIME_LORA_UPDATING=True):

```
curl -X POST localhost:8000/v1/load_lora_adapter -H 'Content-Type: appli
cation/json' \
  -d '{"lora_name":"support_v16","lora_path":"/adapters/support-v16&qu
ot;}'
curl -X POST localhost:8000/v1/unload_lora_adapter -d '{"lora_name":"support_v15"}' -
H 'Content-Type: application/json'
```

3.11 Offline Batch Inference (throughput-optimised)

```
from __future__ import annotations

from vllm import LLM, SamplingParams

llm = LLM(model="meta-llama/Llama-3.1-8B-Instruct", dtype="bfloat16", gpu_memory_util
ization=0.92,
          max_model_len=8192, enable_prefix_caching=True, max_num_seqs=512, max_num_batched_tokens=16384)
params = SamplingParams(temperature=0.0, max_tokens=256)
prompts = [f"Classify the ticket: {t}" for t in open("tickets.txt").read().splitlines
()]
for out in llm.generate(prompts, params):
    print(out.outputs[0].text.strip())
```

Offline/batch jobs should use **large max_num_seqs** and ignore ITL — they trade latency for maximum throughput; run them on separate (often spot) capacity from interactive traffic.

3.12 Prometheus Alert Rules

```
groups:
- name: vllm
  rules:
  - alert: VLLMKVCacheNearFull
    expr: avg(vllm:gpu_cache_usage_perc) > 0.92
    for: 3m
    labels: { severity: page }
    annotations: { summary: "KV cache >92% — preemptions imminent" }
  - alert: VLLMQueueGrowing
    expr: sum(vllm:num_requests_waiting) > 20
    for: 2m
    labels: { severity: page }
  - alert: VLLMHighTTFT
    expr: histogram_quantile(0.95, sum by (le) (rate(vllm:time_to_first_token_seconds_bucket[5m]))) >
1.0
    for: 5m
    labels: { severity: ticket }
  - alert: VLLMPreemptions
    expr: rate(vllm:num_preemptions_total[5m]) > 0.5
    for: 5m
    labels: { severity: ticket }
  - alert: VLLMPrefixCacheHitRateDrop
    expr: rate(vllm:prefix_cache_hits_total[10m]) / rate(vllm:prefix_cache_queries_total[10m]) < 0.3
    for: 15m
    labels: { severity: ticket }
```

(Metric names shift between vLLM releases — verify against /metrics of your version.)

---

4. Hard-Learned Production Failures & Edge Cases

Incident 1 — KV cache exhaustion → preemption thrash → latency cliff

Symptom. At 14:00 daily, p99 E2E jumped from 5 s to 90 s; throughput *fell* while GPU showed 100 % utilisation.

Root-cause analysis. 1. A batch ingestion job (long 20k-token prompts) shared the interactive deployment. 2. gpu_cache_usage_perc hit 100 %. vLLM preempted running sequences via **recompute**: it evicted their KV and later re-ran the prefill — work done twice. 3. Preempted long sequences re-entered the queue, evicted others, causing cascading recomputation (livelock-like behaviour). Throughput of useful tokens collapsed.

Remediation. - Separate pools: interactive (max-model-len 8192) vs batch (large context, max-num-seqs tuned, spot nodes). - Lower --max-num-seqs to what the KV pool sustains; enable FP8 KV; add vllm:num_preemptions_total alert. - Gateway admission control by *estimated KV tokens* in flight, not request count:

```
class KVBudget:
    def __init__(self, capacity_tokens: int, headroom: float = 0.85) -> None:
        self.cap = int(capacity_tokens * headroom); self.used = 0
    def try_acquire(self, prompt_tokens: int, max_out: int) -> bool:
        need = prompt_tokens + max_out
        if self.used + need > self.cap:
            return False            # 429 + Retry-After; or queue with priority
        self.used += need; return True
    def release(self, prompt_tokens: int, max_out: int) -> None:
        self.used -= prompt_tokens + max_out
```

Incident 2 — Rolling deploy dropped streaming responses

Symptom. Every deploy produced a burst of truncated chat responses and 502s.

Root cause. Default terminationGracePeriodSeconds=30 while streams last up to 90 s; SIGTERM killed vLLM mid-stream; Service endpoints were removed *concurrently* with SIGTERM so new requests still hit the terminating pod.

Remediation. preStop: sleep 20, grace period ≥ max stream duration + buffer (120 s), maxUnavailable: 0, readiness probe flips to NotReady on SIGTERM, client-side retry on idempotent connect failures only, and a PodDisruptionBudget. Pre-warm new pods (model load + CUDA graph capture takes 1–3 minutes) via startupProbe so traffic arrives only when ready.

Incident 3 — NCCL hangs and silent TP deadlock on multi-GPU node

Symptom. A 4×GPU TP deployment froze: requests accepted, zero tokens, no errors, GPU memory static.

Root cause. A single GPU hit an uncorrectable ECC error (Xid 48/79) and its rank stopped participating; other ranks blocked in all-reduce indefinitely. Health endpoint returned 200 (the HTTP process was alive).

Remediation. - Deep health check: a periodic canary request (1 token) with a 5 s deadline; liveness probe hits a path that performs real inference. - Export DCGM metrics (DCGM_FI_DEV_XID_ERRORS) and cordon the node on Xid; set NCCL_ASYNC_ERROR_HANDLING=1 and --distributed-executor-backend timeouts; use torch.distributed watchdog envs. - Prefer smaller TP degree plus replicas for fault isolation (one bad GPU takes down TP-1 replica, not a TP-8 mega-replica).

Incident 4 — Speculative decoding made production slower

Symptom. Enabling a 1B draft model for a 70B target increased average latency 15 % during the day.

Root cause. Daytime batches were large (b≈48): the system was already compute-saturated, so verification of rejected speculative tokens consumed real compute; acceptance rate on creative, high-temperature chat was ~0.45. At night (low batch) it helped 1.8×.

Remediation. Make speculation **adaptive**: enable it only below a batch-size threshold (vLLM has dynamic speculation / run separate low-latency pool), monitor spec_decode_draft_acceptance_rate; use n-gram speculation for RAG/summarisation where output copies input (acceptance > 0.7), and stay off for high-temperature generation.

Incident 5 — Prefix cache poisoning by per-request noise

Symptom. Prefix hit-rate ~0 % despite a 2k-token shared system prompt.

Root cause. The framework prepended a unique request_id and timestamp to the system message; block hashes depend on all preceding tokens so no block matched. (Same class of issue as Module 03 Incident 5.)

Remediation. Move volatile fields to the end; add a CI test that renders two requests and asserts their first N tokens are identical.

Incident 6 — Tokenizer/Chat template mismatch between training and serving

Symptom. Model answered correctly via HF generate but produced rambling in vLLM.

Root cause. vLLM applied the model's default chat template; the fine-tune used a custom template (different system-turn markers and EOS token). Missing eos_token_id in generation_config.json meant generation never stopped at <|eot_id|>.

Remediation. Package tokenizer_config.json (with chat_template) and generation_config.json with the model; pass --chat-template explicitly; add a golden test comparing rendered prompt strings and stop tokens across train and serve code paths.

---

5. Senior/Staff LLMOps Interview Questions

Scenario 1 — "Explain PagedAttention and why it improves throughput."

Junior/Mid Trap Answer: "It makes attention faster by using a better kernel."

Staff Production Answer: "PagedAttention is primarily a *memory management* innovation, not a faster attention formula. Before it, systems reserved a contiguous max-length KV buffer per request, wasting 60–80 % of KV memory to internal/external fragmentation. PagedAttention splits the KV cache into fixed-size blocks and uses a per-sequence block table (like OS virtual memory) so blocks are allocated on demand and can be non-contiguous. That cuts waste to under one block per sequence, so many more sequences fit in the batch — and because decode throughput scales almost linearly with batch size while

memory-bound, throughput rises 2–4× or more. It also enables copy-on-write sharing (parallel sampling, beam search), automatic prefix caching via block hashing, and cheap preemption/swap. The cost is a gather through the block table in the attention kernel — a few percent — which is dwarfed by the batching gain."

Scenario 2 — "p99 ITL spikes whenever a user sends a long document. Diagnose and fix."

Junior/Mid Trap Answer: "Add more GPUs so the load is spread out."

Staff Production Answer: "That's head-of-line blocking: a monolithic 30k-token prefill occupies a full iteration (~1 s of compute) and every concurrent decode waits, so all users see an ITL spike. More GPUs reduces frequency, not the mechanism. Fixes: enable **chunked prefill** and size max_num_batched_tokens (say 2–4k) so decodes piggyback on every iteration — I'd sweep chunk sizes and plot p99 ITL vs TTFT of long requests. Beyond that: route long-context requests to a dedicated pool (or use prompt-length-based priority), cap max_model_len for the interactive tier, use prefix caching to skip repeated documents, and at larger scale consider **disaggregated prefill/decode** so prefill bursts can't interfere with decode. Verify via ITL histograms split by whether a prefill was co-scheduled."

Scenario 3 — "Choose between vLLM and TensorRT-LLM for a 500-GPU fleet."

Junior/Mid Trap Answer: "TensorRT-LLM is NVIDIA's, so it's always faster; use it."

Staff Production Answer: "TRT-LLM can deliver, as a rule of thumb that varies by model and workload (10–40 % is illustrative; benchmark yourself), better throughput/latency on NVIDIA via compiled kernels, FP8/FP4 paths and in-flight batching, and at 500 GPUs a 20 % gain is ~100 GPUs — worth real engineering. But costs: engine builds per GPU arch/shape, slower model onboarding (new architectures arrive in vLLM first), tighter coupling to NVIDIA, and harder debugging. I'd run a structured bake-off on production traces: goodput under SLO (TTFT p95, ITL p95), cost per million tokens, operational overhead (build/deploy time, time to support a new model). A typical outcome: TRT-LLM (or Dynamo with TRT-LLM backend) for the 2–3 stable, high-volume models that account for 80 % of spend; vLLM for the long tail, experiments, multi-LoRA and fast-moving models. Both sit behind the same OpenAI-compatible gateway so engines are swappable."

Scenario 4 — "How do you autoscale LLM serving?"

Junior/Mid Trap Answer: "HPA on CPU or GPU utilisation at 70 %."

Staff Production Answer: "GPU utilisation is a poor signal — it's 'any kernel active', and CPU is irrelevant. Saturation shows up as **queue depth** (num_requests_waiting), **KV-cache usage**, and rising TTFT. I'd scale on those through KEDA/Prometheus with a target such as waiting-per-replica ≤ 2 or KV ≤ 80 %. Cold start is the catch: loading 16–140 GB weights plus CUDA-graph capture takes 1–5 min, so reactive scaling lags spikes. Mitigations: keep a warm baseline sized for p95 load; predictive/scheduled scaling for known peaks; fast model loading (local NVMe cache, safetensors + tensorizer/RunAI streamer, or model baked into the image); overflow routing to a smaller/faster model or to a serverless provider when the queue exceeds a threshold; and gateway-level admission control to protect SLOs while new replicas warm up. Scale-in is slow (cooldown 5+ min) to avoid flapping."

Scenario 5 — "When does speculative decoding help and when does it hurt?"

Junior/Mid Trap Answer: "It always speeds up generation because the draft model is faster."

Staff Production Answer: "Speedup = (1–α^{γ+1}) / ((1–α)(γc+1)); it depends on acceptance rate α, draft length γ, draft cost ratio c. It exploits that decode is memory-bound — verifying γ tokens costs about one weight read — so it's great at low batch sizes and for low-entropy outputs (code, JSON, RAG summarisation, edits) where α is 0.7–0.9, giving 2–3×. It hurts at high batch where the GPU is already compute-saturated (verification of rejected tokens burns real FLOPs), with high-temperature creative generation (α < 0.5), or with a poorly-matched draft. It's lossless in distribution, so quality isn't the risk — efficiency is. I'd enable it on a latency-sensitive low-QPS tier, prefer n-gram/EAGLE for RAG workloads, track acceptance rate in metrics, and make it adaptive to load."

Scenario 6 — "Design multi-tenant serving with 300 customer-specific fine-tunes."

Junior/Mid Trap Answer: "Deploy one model replica per customer."

Staff Production Answer: "300 × 16 GB replicas is absurdly wasteful when most are idle. Fine-tune each tenant with LoRA on a shared base and serve with **multi-LoRA** in vLLM: base weights resident once; adapters (~100–300 MB) in a tiered cache — --max-loras on GPU, --max-cpu-loras in RAM, the rest in object storage with dynamic load via the runtime LoRA API. The gateway maps tenant → adapter name, enforces per-tenant quotas, and uses consistent hashing on tenant ID so a tenant's adapter stays warm on a subset of replicas (improving cache hit and prefix cache reuse). Concerns: adapter load latency on cold tenants

(pre-warm top-N), SGMV kernel overhead with many distinct adapters in one batch (limit by max-loras), noisy neighbours (per-tenant rate limits and token budgets), strict isolation of KV/prefix cache across tenants for data-leak prevention (cache salt per tenant), and evaluation per adapter before activation. Very large tenants with high traffic get a dedicated merged-weights deployment."

Scenario 7 — "TP=8 across 8 PCIe GPUs is slower than TP=2. Why?"

Junior/Mid Trap Answer: "More GPUs should be faster; maybe a driver issue."

Staff Production Answer: "TP performs two all-reduces per layer per step, on the critical path. Over NVLink that's negligible; over PCIe (~25 GB/s effective, higher latency, and no direct P2P across root complexes) communication dominates decode, which is a few ms per step. At TP=8 each GPU also does less compute per GEMM, lowering arithmetic efficiency. So for PCIe boxes I'd minimise TP (use quantization to fit on 2 GPUs), run multiple TP=2 replicas (data parallel) for throughput, or use pipeline parallel across slower links — accepting bubble overhead mitigated by many concurrent microbatches. I'd verify with nvidia-smi topo -m and NCCL tests (all_reduce_perf) before deciding."

Scenario 8 — "TTFT is great at 10 RPS and awful at 18 RPS; GPU 'util' is only 85 %."

Junior/Mid Trap Answer: "85 % isn't saturated, so it must be a network issue."

Staff Production Answer: "Queueing theory: waiting time grows like $\rho/(1-\rho)$; around 85–90 % effective load, delay explodes non-linearly. The 85 % 'GPU util' isn't the true load factor — effective load is determined by the bottleneck resource (KV pages or scheduler token budget). I'd check num_requests_waiting, KV usage and preemption counters; likely KV is nearly full so new requests wait for blocks. Solutions: raise capacity (FP8 KV, shorter max-model-len, more replicas), reduce per-request footprint (prefix caching, shorter prompts via reranking), or add priority classes and admission control with 429 to keep tail latency bounded instead of letting everyone slow down. I'd target $\rho \leq 0.7$ for interactive SLOs and use load-test curves to set autoscaling thresholds."

Key Takeaways

- vLLM is the default; TRT-LLM wins on fixed high-volume NVIDIA fleets; Ollama is for dev/edge.
- PagedAttention = memory management; continuous batching = scheduling; chunked prefill = fairness; speculation = latency at low batch.
- Scale on **queue depth/KV usage**, not GPU utilisation. Plan for 1–5 minute cold starts.
- Measure **goodput under SLO** with an open-loop load generator using production-shaped prompts.
- Separate interactive vs batch traffic; keep TP within NVLink domains; test health with real inference.

5.6 Prefix and KV-Cache Management at Scale: Cache-Aware Routing and KV Offload

Conceptual Deep Dive

Prefix caching reuses the KV blocks of a shared prompt prefix instead of recomputing prefill. vLLM's automatic prefix caching hashes each full block together with the hash of everything before it, so a block is reusable only if the whole prefix up to it matches; SGLang's RadixAttention keeps a radix tree over token sequences and shares the longest matching path. Both give the same operational lesson: the hit rate is decided by **prompt layout** (stable content first, volatile content last) and by **which replica holds the blocks**, not by the engine flag. Only full blocks count, so a larger block size wastes the partial tail and coarsens sharing.

With N replicas behind a round-robin balancer each replica sees a 1/N slice of every prefix, so hit rate collapses as the fleet grows. **Cache-aware routing** hashes the leading N tokens (or a session or tenant id) with consistent or rendezvous hashing so similar prefixes meet on one replica. The cost is load imbalance when a few prefixes are hot; bounded-load variants spill to the next-ranked replica once a replica exceeds c times the average load, trading a little hit rate for a controllable tail. Consistent hashing also limits cache movement when replicas are added or removed. **KV offload** to CPU RAM, NVMe or a remote store (LMCache-style projects exist; check current docs) pays off when transfer time is below recompute time: break-even is KV bytes / link bandwidth versus tokens / prefill speed. KV bytes per token are roughly 2 x layers x KV heads x head dim x dtype bytes, so a long prefix on a fast link usually wins, a short one on NVMe usually does not.

Eviction defaults to LRU; add priority classes (pin system prompts of paying tenants) so one batch tenant cannot flush them. Shared caches leak timing side channels (a fast first token reveals that someone else sent that prefix) and can be poisoned by crafted prefixes (Module 05 Incident 5, legacy); namespace the cache key per tenant when isolation matters, accepting lower

sharing.

Metric	Why
Prefix hit rate (tokens, not requests)	Primary health signal
Prefill tokens saved	Converts to GPU-seconds and cost
Eviction rate, cache memory share	Capacity pressure vs batch headroom
TTFT split by hit / miss	Proves the cache pays for itself

Edge cases: a rolling deploy empties every replica cache, so TTFT spikes until traffic repopulates it; replay recent popular prefixes against new replicas before shifting traffic. A change in replica count reshuffles naive modulo hashing; rendezvous or consistent hashing moves only about 1/N of keys.

Cross-Connection Scenarios

- Module 05 section 1.3 (legacy) block geometry sets the sharing granularity used here.
- Module 05 Incident 5 (legacy) is the poisoning case that motivates namespacing.
- 4.6 KV-cache quantisation shrinks bytes per token, shifting the offload break-even.
- 5.7 long-context serving depends on prefix reuse to keep prefill affordable.
- Module 10 dashboards should split TTFT by cache hit and miss.

Production Anti-Patterns

- Round-robin load balancing in front of a prefix-cache-heavy workload: each replica caches 1/N of every prefix and the hit rate falls as you scale out.
- Per-request timestamps or UUIDs at the top of the prompt: the first block never matches, so nothing after it can be reused.
- One shared prefix cache across tenants with no isolation review: timing leaks and cache poisoning cross tenant boundaries.

Minimal Working Code

A block-hash prefix cache with LRU capacity; the demo compares block sizes on shared system prompts plus unique suffixes (500-token prompts do not fill large blocks).

```
import random
from collections import OrderedDict

class PrefixCache:
    def __init__(self, capacity_blocks, block):
        self.cap, self.block, self.lru = capacity_blocks, block, OrderedDict()
        self.hit = self.total = 0

    def request(self, tokens):
        b, h, keys = self.block, 0, []
        for i in range(len(tokens) // b):
            h = hash((h, tuple(tokens[i * b:(i + 1) * b])))
            keys.append(h)
        lead = 0
        for k in keys:
            if k not in self.lru:
                break
            lead += 1
        for k in keys:
            self.lru[k] = True
            self.lru.move_to_end(k)
        while len(self.lru) > self.cap:
            self.lru.popitem(last=False)
        self.hit += lead * b
        self.total += len(tokens)

def workload(n=2000, seed=0):
    rng = random.Random(seed)
    for _ in range(n):
        sid = min(int(rng.expovariate(0.7)), 7)
```

```
        sysp = [sid * 1000 + j for j in range(500)]
        yield sysp + [rng.randrange(10**6, 10**7) for _ in range(rng.randint(100, 300))]

for block in (16, 64, 256):
    c = PrefixCache(capacity_blocks=4000 // block * 4, block=block)
    for p in workload():
        c.request(p)
    print(block, round(c.hit / c.total, 3))
```

A rendezvous-hash router with bounded load; the demo sweeps the load-bound factor c and prints prefix hit rate against load imbalance, with round-robin as the baseline.

```
import hashlib, random

def hr(key, r):
    return int(hashlib.md5(f"{key}|{r}".encode()).hexdigest(), 16)

def simulate(n_rep=8, c=None, n=4000, n_keys=300, seed=1, rr=False):
    rng = random.Random(seed)
    weights = [1 / (i + 1) for i in range(n_keys)]
    loads, seen, hits = [0] * n_rep, [set() for _ in range(n_rep)], 0
    for i in range(n):
        key = rng.choices(range(n_keys), weights)[0]
        if rr:
            r = i % n_rep
        else:
            order = sorted(range(n_rep), key=lambda x: -hr(key, x))
            cap = float("inf") if c is None else c * (sum(loads) + 1) / n_rep
            r = next((x for x in order if loads[x] + 1 <= cap), min(range(n_rep), key=loads.__getitem_
_))
        hits += key in seen[r]
        seen[r].add(key)
        loads[r] += 1
    return hits / n, max(loads) / (n / n_rep)

for label, kw in [("round-robin", dict(rr=True)), ("c=1.25", dict(c=1.25)),
                  ("c=1.5", dict(c=1.5)), ("c=2.0", dict(c=2.0)), ("pure",
dict())]:
    hit, imb = simulate(**kw)
    print(f"{label:12s} hit={hit:.3f} max/mean load={imb:.2f}")
```

Interview Angle

Q: Prefix hit rate is 10% although every prompt shares a 4k-token system prompt. Walk through diagnosis. **A:** Check layout first: any timestamp, user id or randomised few-shot ordering before the shared text breaks the block-hash chain. Then check routing (round-robin spreads the prefix over all replicas), then eviction (cache memory share and eviction rate under batch pressure), then tokenisation or template drift between services. Confirm by replaying one prompt twice against one replica: the second TTFT should drop sharply. Target: token-weighted hit rate close to shared-prefix tokens divided by total.

Q: Cache-aware routing vs least-loaded routing: how do you choose? **A:** Measure the shared-prefix ratio. If most prompt tokens are shared (RAG with fixed instructions, agents with long tool schemas), affinity saves more prefill than balance costs; use bounded-load rendezvous with c around 1.25-1.5 and watch p99 TTFT per replica. If prompts are mostly unique, least-loaded wins. Decision check: A/B on TTFT p95 and GPU-seconds per 1M requests.

Q: When does KV offload beat recomputation? **A:** When KV bytes / effective link bandwidth is below prefill time for those tokens, plus lookup overhead. Long prefixes on PCIe-attached host RAM usually win; short ones, or slow NVMe and remote stores under contention, lose. Compute the break-even per model (bytes per token is fixed), then validate by measuring TTFT with offload on and off at your real prefix-length distribution.

5.7 Serving MoE and Long-Context Models

Conceptual Deep Dive

A mixture-of-experts model stores E experts per MoE layer but routes each token to k of them. **Memory scales with total parameters; per-token compute scales with active parameters.** A 400B-total, 40B-active model needs weights for 400B resident, but each token costs roughly the FLOPs of a 40B model. Decode is usually bandwidth-bound, and at moderate batch the tokens in a step route to many different experts. The expected number of distinct experts touched per layer is E x (1 - (1 - k/E)^B): with E=64, k=2, B=32 it is about 64 x (1 - 0.969^32), roughly 41, so most of the expert weights are read every step. Decode therefore does not become 10x faster than a dense 400B model; the gain shows up mainly at small batch and in prefill, which is compute-bound.

Expert parallelism (EP) places different experts on different GPUs and uses all-to-all communication to dispatch tokens and gather results, twice per MoE layer. It is typically combined with TP or DP for the attention layers. Failure modes: load imbalance (the slowest expert shard sets step time), capacity factors that drop or overflow tokens, and routing hot-spots when traffic is skewed toward one domain or language. Mitigations are expert replication, load-balancing losses at training time (not controllable at serving), and monitoring per-expert token counts.

For long context, KV memory is linear in tokens per sequence while prefill attention compute grows quadratically, so long requests dominate both memory and latency. Levers: chunked prefill (Module 05 section 1.5, legacy), context or sequence parallelism and ring attention across GPUs (concept), and model-specific variants such as sliding-window layers or compressed KV (check the model card, do not assume). KV quantisation (4.6) and prefix reuse (5.6) cut cost further; admission control by context length keeps one request from consuming the pool.

Operationally, separate pools by context-length class so a few 128k requests cannot starve short chat (extends Module 05 section 1.9 on disaggregation). Capacity planning must use the **context-length distribution**, not the mean: the tail sets concurrency limits and memory headroom.

Edge cases: a fleet mixing MoE and dense models has different scaling signals (MoE is memory- and communication-bound, dense is closer to compute-bound); one autoscaling rule will mis-size one of them. A long-context request arriving during a burst can trigger a preemption storm (Module 05 Incident 1, legacy); reserve headroom per class and cap admitted tokens.

Cross-Connection Scenarios

- Module 05 section 1.8 (legacy) gives the TP/PP/EP vocabulary used here.
- 4.6 KV quantisation and 5.6 prefix reuse are the main long-context cost levers.
- Module 11 cost models need the length distribution from 5.8 workloads.
- Module 05 Incident 1 (legacy) is the preemption failure triggered by long requests.

Production Anti-Patterns

- Sizing MoE GPU memory from active parameters: the model will not load, or leaves no room for KV.
- One pool for 2k and 128k contexts under a single SLO: long requests inflate queueing for everyone.
- Benchmarking long context with synthetic filler that is highly prefix-shared: prefix caching hides the real prefill and memory cost.

Minimal Working Code

A numpy MoE calculator: weight bytes, bandwidth-bound decode throughput including the expected number of distinct experts touched, compared with a dense model of the same total size. Hardware figures are placeholders.

```
import numpy as np

def moe_row(E, k, B, total=400e9, expert_frac=0.9, bytes_per_param=1.0, bw=24e12):
    """bw = aggregate memory bandwidth (placeholder, bytes/s across the serving group).&qu
ot;""
    distinct = E * (1 - (1 - k / E) ** B)
    dense_part = total * (1 - expert_frac)
    expert_part = total * expert_frac
    active = dense_part + expert_part * k / E
    read = (dense_part + expert_part * distinct / E) * bytes_per_param
    moe_tps = B / (read / bw)
    dense_tps = B / (total * bytes_per_param / bw)
    return distinct, active / 1e9, total * bytes_per_param / 1e9, moe_tps, moe_tps / dense_tps

print(f"{'E':>4}{'k':>3}{'B':>5}{'distinct':>9}{'active_B':>9}{'weightGB':>9}{'tok
/s':>10}{'vs dense':>9}")
for E, k, B in [(64, 2, 1), (64, 2, 32), (64, 2, 256), (256, 8, 1), (256, 8, 32), (256, 8, 256)]:
    d, a, w, t, r = moe_row(E, k, B)
```

```
    print(f"{E:4d}{k:3d}{B:5d}{d:9.1f}{a:9.1f}{w:9.0f}{t:10.0f}{r:9.2f}")
```

A long-context capacity planner: KV bytes per sequence with GQA and a fraction of sliding-window layers, concurrent sequences per context length, and a FLOPs-based prefill estimate. All hardware numbers are stated assumptions.

```
def kv_per_seq(ctx, layers=80, kv_heads=8, head_dim=128, dtype=2, sw_f
rac=0.0, window=4096):
    per_tok_layer = 2 * kv_heads * head_dim * dtype
    local = int(layers * sw_frac)
    return per_tok_layer * ((layers - local) * ctx + local * min(ctx, window))

def prefill_s(ctx, params=70e9, layers=80, d_model=8192, peak=1e15, mfu=0.4, gpus=4):
    flops = 2 * params * ctx + 2 * layers * d_model * ctx ** 2   # weights + causal attention
    return flops / (peak * mfu * gpus)

kv_budget = 4 * 80e9 * 0.9 - 70e9 * 2   # 4 GPUs x 80 GB x 90% minus fp16 weights (assumption)
print(f"{'ctx':>7}{'KV GB/seq':>10}{'max seqs':>9}{'(sw 50%)':>9}{'prefill s':>10}&qu
ot;)
for ctx in (2_048, 8_192, 32_768, 131_072):
    a, b = kv_per_seq(ctx), kv_per_seq(ctx, sw_frac=0.5)
    print(f"{ctx:7d}{a / 1e9:10.2f}{kv_budget / a:9.1f}{kv_budget / b:9.1f}{prefill_s(ctx):10.2f}&qu
ot;)
```

Interview Angle

Q: A 400B-parameter MoE has 40B active parameters. Does it need 40B or 400B worth of GPU memory, and why is decode not 10x faster than dense 400B? **A:** 400B: every expert must be resident because any token may route to any of them. Decode at batch B reads about E(1-(1-k/E)^B) experts per layer, so at batch 32 most experts are touched and bytes read approach the full model; speedup only approaches 10x at batch 1. Check: log distinct experts per step and tokens/s versus batch against a dense baseline.

Q: How do you serve 8k and 128k requests on the same cluster? **A:** Split pools by length class (or disaggregate prefill), give each its own SLO and autoscaler, use chunked prefill and admission control on the long pool, and size from the context-length distribution. Trade-off: lower utilisation from fragmentation versus protected short-chat TTFT. Check p99 TTFT of short requests while a long-request burst is replayed.

Q: What is expert parallelism and what breaks when traffic is skewed? **A:** Experts are sharded across GPUs with all-to-all dispatch per MoE layer. Skewed routing overloads some shards, so step time follows the slowest and tokens may be dropped past capacity. Mitigate by replicating hot experts, adjusting capacity factors and rebalancing placement. Check: per-expert token counts and step-time variance by traffic mix.

5.8 Benchmarking Inference Honestly: Workloads, Open-Loop Load and Goodput

Conceptual Deep Dive

Most published inference numbers are produced by **closed-loop** load: a fixed number of clients each send a request, wait for the response, then send the next. When the server stalls, the clients stall with it and stop generating the requests that would have queued, so the stall appears in a handful of samples. This is **coordinated omission**: the measurement coordinates with the system under test and omits the very requests that would have shown the damage. Production traffic is **open-loop**: users arrive independently of how busy you are, so a 20-second stall delays every request that arrives during it. Use an open-loop generator (arrivals scheduled from a Poisson or bursty process) and measure latency from the *intended* send time, not the actual one.

Realism requires matching the workload: sample or replay real input and output length distributions (not a single fixed length), the shared-prefix ratio, burstiness, and streaming versus non-streaming clients. Random prompts defeat prefix caching and understate production performance; identical prompts overstate it. Reach steady state before measuring: warm-up, discard the ramp, and run long enough that the tail percentiles are populated (p99 needs thousands of requests; p99.9 far more). Report intervals, for example by bootstrap, and repeat runs.

The headline metric is **goodput**: requests per second that meet both the TTFT and inter-token latency (ITL) SLOs, not raw tokens per second. Sweep request rate, plot goodput and p99 latency, and find the knee where latency leaves the SLO; capacity is the

rate just below it. Peak tokens/s at batch 256 describes a regime your users will never see within their SLO.

Fair engine comparisons need identical hardware, driver and engine versions, flags, quantisation, max model length, sampling parameters and seeds, with the full config published. Tooling examples: vLLM's benchmark scripts, GuideLLM, NVIDIA genai-perf (check current docs for flags). A report template: hardware, versions, workload, SLOs, results table, knee, cost per 1M tokens at SLO, caveats.

Pitfall	Effect
Closed loop	Understates tail latency
Fixed-length prompts	Misstates batching and memory behaviour
Load generator on a small machine	Measures the client, not the server
Single run	No variance estimate

Edge cases: a client machine saturated on CPU, TLS or tokenisation becomes the bottleneck and caps throughput; check generator CPU and send-time lag. Prompt randomness cuts both ways, so state the shared-prefix ratio explicitly and test at the production value.

Cross-Connection Scenarios

- Module 05 section 3.9 (legacy) is a load generator; extend it to open-loop with intended-send-time accounting.
- 5.6 hit rates only appear if the benchmark workload reproduces real prefix sharing.
- 5.7 length distributions feed workload sampling here.
- Module 11 cost per 1M tokens must be computed at the SLO knee, not at peak throughput.
- 14.5 supplies the bootstrap and noise-floor methods for benchmark confidence intervals.

Production Anti-Patterns

- Quoting peak tokens/s at batch 256 as "the speed": it ignores TTFT and ITL SLOs and overstates capacity.
- Comparing engines with different quantisation or max-length settings: the result measures configuration, not engine quality.
- A single run with no variance estimate: differences inside run-to-run noise get reported as wins.

Minimal Working Code

A discrete-event single-server queue contrasting closed-loop and open-loop measurement of p99 latency under occasional long stalls.

```
import random

def p(xs, q):
    xs = sorted(xs)
    return xs[min(len(xs) - 1, int(q * len(xs)))]

rng = random.Random(7)
n = 5000
service = [20.0 if rng.random() < 0.003 else rng.uniform(0.3, 0.7) for _ in range(n)]

closed = list(service)                      # client waits, so no queueing is ever observed

t, free, open_lat = 0.0, 0.0, []
for s in service:
    t += rng.expovariate(1 / 1.0)           # Poisson arrivals, mean gap 1.0 (utilisation about 0.5)
    start = max(t, free)
    free = start + s
    open_lat.append(free - t)               # latency from intended arrival time

print("closed-loop p99:", round(p(closed, 0.99), 2))
print("open-loop   p99:", round(p(open_lat, 0.99), 2))
assert p(open_lat, 0.99) > 5 * p(closed, 0.99)
```

Sampling request lengths from an empirical histogram, percentiles with bootstrap CIs, and a goodput function over TTFT and ITL arrays (the latency model is synthetic and illustrative).

```
import numpy as np

rng = np.random.default_rng(0)
lens = np.array([128, 512, 2048, 8192])
counts = np.array([500, 300, 150, 50])
in_len = rng.choice(lens, size=4000, p=counts / counts.sum())
ttft = 0.05 + 0.0002 * in_len + rng.lognormal(-3, 0.6, size=in_len.size)   # seconds, synthetic
itl = 0.02 + rng.gamma(2.0, 0.004, size=in_len.size)                       # mean ITL per request

def boot_ci(x, q, b=500, alpha=0.05):
    stats = [np.percentile(rng.choice(x, size=x.size), q) for _ in range(b)]
    return np.percentile(stats, [100 * alpha / 2, 100 * (1 - alpha / 2)])

def goodput(ttft, itl, duration_s, slo_ttft=1.0, slo_itl=0.05):
    ok = (ttft <= slo_ttft) & (itl <= slo_itl)
    return ok.sum() / duration_s, ok.mean()

for q in (50, 95, 99):
    lo, hi = boot_ci(ttft, q)
    print(f"TTFT p{q}: {np.percentile(ttft, q):.3f}  95% CI [{lo:.3f}, {hi:.3f}]")
rate, frac = goodput(ttft, itl, duration_s=600)
print(f"goodput {rate:.2f} req/s, {frac:.1%} of requests meet both SLOs")
```

Interview Angle

Q: A vendor says 10k tokens/s. What do you ask before believing it? **A:** Hardware and count, model and quantisation, batch or concurrency, input and output lengths, whether it is open- or closed-loop, the TTFT and ITL at that point, prefix-cache assumptions, and whether it is output or total tokens. Trade-off: peak throughput versus SLO-bound capacity. Check: rerun on our length distribution with open-loop load and report goodput and cost per 1M tokens at SLO.

Q: Design a benchmark to choose between two serving engines for a RAG chat product. **A:** Replay sampled production prompts (real lengths, shared-prefix ratio), open-loop Poisson plus a burst profile, streaming clients, identical hardware, quantisation and max length. Sweep rate, record TTFT and ITL, find each engine's goodput knee, repeat 3-5 times with bootstrap intervals. Decide on cost per 1M tokens at SLO plus operability. Check: the generator CPU stays below saturation.

Q: What is coordinated omission and how do you avoid it? **A:** The load generator delays sends while waiting on slow responses, so the requests that would have queued are never measured. Avoid it with open-loop scheduling and latency measured from intended send time, or correct afterwards by back-filling the missed arrivals. Check: compare p99 against a closed-loop run; a large gap means the closed-loop number hid queueing.